Entwicklungsdokumentation: Pablo - Desk-Companion mit Sprachinteraktion

Helena Bongartz, Elias Bürgin, Paul Siewert

3. April 2026

Inhaltsverzeichnis

1 Einleitung **2**

2 Anforderungsdefinition **3**

2.0.1 Funktionale Anforderungen . . . 3

2.0.2 Nicht-funktionale Anforderungen . . . 3

3 Entwicklungsprozess **3**

3.1 Konzeption . . . 3

3.2 Iterative Entwicklung . . . 4

3.3 Praktische Herausforderungen . . . 5

4 Produktbeschreibung **6**

4.1 Kernfunktionen . . . 6

4.2 Softwarearchitektur . . . 7

4.2.1 (1) Aktivierung des Systems . . . 8

4.2.2 (2) Spracherkennung (STT) . . . 8

4.2.3 (3) LLM-Verarbeitung . . . 8

4.2.4 (4) Sprachausgabe (TTS) . . . 8

4.3 Hardwareaufbau . . . 8

5 Begründung zentraler Entwicklungsentscheidungen **8**

5.1 Entscheidung für einen physischen Companion statt reinem Softwareassistenten . 8

5.2 Entscheidung für modular erweiterbare API-Struktur . . . 8

5.3 Entscheidung für Raspberry-Pi-basierte Hardware . . . 9

5.4 Entscheidung für Wakeword- und Sprachsteuerung . . . 9

5.5 Entscheidung für eine visuelle Persona . . . 9

5.6 Entscheidung für ein kompaktes Display . . . 9

6 Ergebnis und kritische Einordnung **9**

1 Einleitung

Der Umgang mit digitalen Assistenzsystemen hat sich in den vergangenen Jahren grundlegend verändert. Sprachbasierte Systeme wie Amazon Alexa, Google Assistant und Apple Siri sind heute in Millionen von Haushalten verfügbar und ermöglichen eine freihändige, niedrigschwellige Interaktion im Alltag [1], [2]. Parallel dazu haben Fortschritte im Bereich der großen Sprachmodelle (Large Language Models, LLMs) die Qualität maschinell generierter Antworten erheblich gesteigert, sodass natürlichsprachliche Dialoge zunehmend flüssig und kontextbezogen geführt werden können [3]. Diese Entwicklung hat die Erwartungen an digitale Assistenzsysteme nachhaltig verändert: Nutzende setzen zunehmend voraus, dass Systeme nicht nur einzelne Befehle ausführen, sondern flexibel auf offene Fragestellungen reagieren und sich in individuelle Arbeitsabläufe einfügen.

Trotz der breiten Verbreitung sprachbasierter Assistenten bestehen wesentliche Einschränkungen, die deren Einsatz im persönlichen Umfeld begrenzen. Kommerzielle Systeme sind überwiegend als geschlossene Plattformen konzipiert, deren Funktionsumfang herstellerseitig definiert und nur eingeschränkt anpassbar ist [4]. Darüber hinaus reduziert sich die Interaktion in der Regel auf rein auditive Ein- und Ausgabe, ohne dass eine physische Präsenz oder visuelle Rückmeldung das Nutzungserlebnis ergänzt. Forschungsarbeiten im Bereich der sozialen Robotik zeigen jedoch, dass die physische Verkörperung eines Systems einen positiven Einfluss auf die wahrgenommene Vertrauenswürdigkeit, die emotionale Bindung und die langfristige Nutzungsbereitschaft haben kann [5], [6]. Ein Assistenzsystem, das als greifbares, sichtbares Gegenüber im Alltag wahrgenommen wird, besitzt demnach ein qualitativ anderes Interaktionspotenzial als ein reiner Softwareagent.

An diesem Punkt setzt das vorliegende Projekt an. Pablo ist ein kompakter Desk-Companion-Bot, der sprachbasierte Assistenzfunktionen mit einer physischen Verkörperung in Form eines 3D-gedruckten Gehäuses, eines integrierten Displays und eines beweglichen Gelenks vereint. Im Unterschied zu bestehenden Systemen verfolgt Pablo einen bewusst modularen und offenen Ansatz: Durch eine Plugin-basierte Softwarearchitektur können Funktionsmodule – etwa Notiz- und Kalenderverwaltung, Timer oder Wetterabfragen – bedarfsgerecht ergänzt und individuell angepasst werden, ohne den Kernprozess verändern zu müssen. Die zugrunde liegende Hardware basiert auf einem Raspberry Pi und ermöglicht eine kostengünstige, eigenständige Umsetzung. Durch die Kombination aus natürlicher Sprachinteraktion, visueller Darstellung und physischer Ausrichtung zur sprechenden Person soll Pablo als persönlicher Companion wahrgenommen werden, der sich nahtlos in den Arbeitsplatz oder das häusliche Umfeld einfügt.

Ziel dieses Projekts ist die Entwicklung eines funktionalen Minimum Viable Products (MVP), das die technische Machbarkeit und das Interaktionskonzept eines solchen Desk-Companion-Bots demonstriert. Die vorliegende Dokumentation beschreibt den vollständigen Entwicklungsprozess – von der Konzeption über die Hard- und Softwareentwicklung bis hin zur Integration und Evaluierung des Gesamtsystems. Dabei orientiert sich die Entwicklung methodisch an der VDI 2221/2206 und gliedert sich in klar definierte Projektphasen.

2 Anforderungsdefinition

Aus der in der Einleitung beschriebenen Zielsetzung und den angestrebten Kernfunktionen wurden funktionale sowie nicht-funktionale Anforderungen an das System abgeleitet.

2.0.1 Funktionale Anforderungen

Das System muss über ein sprachbasiertes Ein- und Ausgabesystem verfügen, das gesprochene Anfragen erfassen und verarbeitete Antworten als Sprache ausgeben kann. Eine Wakeword-Erkennung ist erforderlich, um das System gezielt aktivieren zu können, ohne dass eine dauerhafte Verarbeitung aller Audioeingaben stattfindet. Darüber hinaus muss das System eine API-basierte Erweiterbarkeit bieten, über die zusätzliche Funktionsmodule in das bestehende System integriert werden können. Auf dem integrierten Display soll ein visueller Avatar dargestellt werden, der den aktuellen Systemzustand widerspiegelt und die Interaktion durch eine grafische Benutzeroberfläche ergänzt.

2.0.2 Nicht-funktionale Anforderungen

Die Bedienung des Systems muss intuitiv gestaltet sein, sodass keine technischen Vorkenntnisse für die Nutzung erforderlich sind. Der Systemaufbau soll modular konzipiert werden, um einzelne Komponenten unabhängig voneinander austauschen oder erweitern zu können. Die verwendete Hardware muss kompakt dimensioniert sein, damit das Gesamtsystem als Schreibtischgerät eingesetzt werden kann. Die Integration der einzelnen Hard- und Softwarekomponenten muss robust umgesetzt werden, um einen zuverlässigen Dauerbetrieb zu gewährleisten. Das System soll echtzeitnahe Reaktionszeiten aufweisen, damit die sprachliche Interaktion als natürlich und flüssig wahrgenommen wird. Die gesamte Interaktion, einschließlich der sprachlichen Ausgabe, muss verständlich und nachvollziehbar gestaltet sein.

3 Entwicklungsprozess

Die Entwicklung von Pablo folgte einem iterativen Vorgehen, das sich methodisch an der VDI 2221 /2206 orientiert. Ausgehend von einer initialen Konzeptionsphase wurde das System schrittweise aufgebaut, wobei in jeder Iteration Hardware- und Softwarekomponenten parallel weiterentwickelt und integriert wurden. Im Folgenden werden die Konzeption, die einzelnen Entwicklungsphasen sowie die dabei aufgetretenen praktischen Herausforderungen beschrieben.

3.1 Konzeption

Am Ausgangspunkt der Entwicklung stand die Beobachtung, dass im Alltag häufig kleine Aufgaben, spontane Ideen und Erinnerungen anfallen, die in der Regel unstrukturiert bleiben. Bestehende digitale Assistenzsysteme bieten zwar Unterstützung bei einzelnen Aufgaben, sind jedoch überwiegend als rein softwarebasierte oder in geschlossene Ökosysteme eingebettete Lösungen konzipiert. Die erste Ideenfindung zielte daher auf ein System, das diese Lücke durch eine physisch präsente, offene und erweiterbare Lösung schließt.

Im Rahmen der Zielgruppenanalyse wurde festgestellt, dass insbesondere Personen im häuslichen oder studentischen Umfeld von einem kompakten Schreibtischassistenten profitieren können, der ohne Smartphone oder Computer bedienbar ist. Daraus ergab sich die bewusste Entscheidung, Pablo nicht ausschließlich als funktionalen Assistenten zu konzipieren, sondern als *Companion*, also ein System, das durch seine physische Gestalt, seine visuelle Identität und seine Interaktionsweise eine persönliche Bindung ermöglicht. Die Verkörperung als Pixel-Art-Faultier in einem baumstammförmigen Gehäuse unterstreicht diesen Companion-Charakter und schafft eine niedrigschwellige, sympathische Zugangsmöglichkeit.

Auf Grundlage dieser Überlegungen wurden die Kernfunktionen definiert: Wakeword-basierte Aktivierung, sprachliche Interaktion über ein LLM, konkrete Alltagsfunktionen (Notizen, Kalender, Timer, Wetterabfrage) sowie visuelle Rückmeldung über ein integriertes Display. Diese Funktionen bilden den minimalen Umfang des angestrebten MVP und wurden so gewählt, dass sie den modularen Charakter des Systems demonstrieren.

3.2 Iterative Entwicklung

Die Umsetzung von Pablo erfolgte in fünf aufeinander aufbauenden Entwicklungsphasen, die jeweils mit einem definierten Zwischenstand abgeschlossen wurden.

Phase 1: Grundidee und Funktionsrahmen. In der ersten Phase wurde die grundlegende Systemidee konkretisiert und der funktionale Rahmen festgelegt. Zentrale Fragestellungen waren, welche Rolle Pablo im Alltag der Nutzenden einnehmen soll und welche Minimalfunktionen für einen sinnvollen Prototyp notwendig sind. Ergebnis dieser Phase war ein technisches Konzeptdokument, das die Systemgrenzen, die angestrebte Interaktionsform und die geplante Modulstruktur beschreibt.

Phase 2: Hardwareauswahl und Aufbau. In der zweiten Phase wurden die Hardwarekomponenten des Systems ausgewählt und zu einem ersten physischen Aufbau zusammengeführt. Die Auswahl erfolgte unter Abwägung von Kosten, Kompaktheit, Integrationsfähigkeit und Community-Unterstützung. Im Ergebnis wurde eine Systemarchitektur definiert, die eine zentrale Recheneinheit, eine Mikrofoneinheit, ein Display, einen Lautsprecher mit Verstärker sowie einen Servomotor für die mechanische Ausrichtung des Kopfteils umfasst. Die konkreten Komponenten und deren Zusammenspiel werden in Kapitel 4 beschrieben.

Phase 3: Implementierung der Sprachpipeline. In der dritten Phase wurde die zentrale Verarbeitungskette für die sprachliche Interaktion aufgebaut. Diese umfasst vier sequenzielle Stufen: Wakeword-Erkennung, Spracherkennung (Speech-to-Text), inhaltliche Verarbeitung über ein Large Language Model sowie Sprachausgabe (Text-to-Speech). Ziel dieser Phase war eine erste durchgängig lauffähige Systemkette, in der eine gesprochene Anfrage vollständig erfasst, verarbeitet und beantwortet werden kann. Die einzelnen Komponenten wurden bewusst als austauschbare Module implementiert, um eine spätere Anpassung einzelner Stufen zu ermöglichen.

Phase 4: Integration von Funktionsmodulen. Aufbauend auf der funktionierenden Sprachpipeline wurden in der vierten Phase konkrete Alltagsfunktionen als eigenständige Module integriert. Dazu zählen unter anderem Wetterabfragen, Kalendereinträge, Timer und das Erstellen von Notizen. Die Einbindung erfolgt über eine einheitliche Plugin-Schnittstelle, die es dem System ermöglicht, kontextbezogen zwischen allgemeiner Antwortgenerierung und gezielter Modulausführung zu unterscheiden. Diese Architektur bildet die Grundlage für die angestrebte langfristige Erweiterbarkeit des Systems.

Phase 5: Benutzererlebnis und Persona. In der abschließenden Phase lag der Fokus auf dem Benutzererlebnis und der visuellen Identität von Pablo. Das Gehäuse wurde als 3D-gedruckter Baumstamm gestaltet, in dem sämtliche Hardwarekomponenten untergebracht sind. Auf dem Display wird Pablo als animiertes Pixel-Art-Faultier dargestellt, das je nach Systemzustand unterschiedliche visuelle Zustände zeigt. Darüber hinaus wurde die Interaktion hinsichtlich Antwortzeiten, Sprachausgabequalität und visueller Synchronisation optimiert.

3.3 Praktische Herausforderungen

Im Verlauf der Entwicklung traten mehrere technische Herausforderungen auf, die exemplarisch für die Komplexität eingebetteter Systeme mit heterogener Hardware- und Softwarelandschaft stehen.

Ein wiederkehrendes Problem betraf den automatischen Systemstart nach dem Booten des Raspberry Pi. Da mehrere Dienste, namentlich die grafische Benutzeroberfläche, der Audiotreiber und der Pablo-Service, beim Hochfahren parallel initialisiert werden, kam es zu Timing-Konflikten. Insbesondere die Audioeingabe über den ReSpeaker-HAT war nach dem Systemstart nicht immer zuverlässig verfügbar, da der entsprechende ALSA-Treiber erst nach einer systemabhängigen Verzögerung vollständig geladen wurde. Diese Problematik wurde durch eine gezielte Startverzögerung und eine Prüfschleife auf Verfügbarkeit der Audioquelle adressiert.

Eine weitere Herausforderung lag in der Integration der unterschiedlichen Hard- und Softwarekomponenten auf der begrenzten Embedded-Plattform. Die Kombination aus Raspberry Pi, zwei Arduino-Mikrocontrollern, dem Mikrofon-HAT, Display und Audioverstärker erforderte eine sorgfältige Abstimmung der seriellen Kommunikation, der Stromversorgung und der GPIO-Belegung. Fehler in der Verkabelung oder Timing-Abweichungen in der seriellen Kommunikation mit den Arduinos führten wiederholt zu schwer reproduzierbaren Fehlerbildern, die nur durch systematisches Testen und schrittweise Integration gelöst werden konnten.

Schließlich stellte die Balance zwischen technischer Funktionalität und einem angenehmen Nutzungserlebnis eine übergreifende Herausforderung dar. So war die reine Funktionsfähigkeit der Sprachpipeline zwar vergleichsweise früh erreicht, die wahrgenommene Qualität der Interaktion wurde jedoch maßgeblich durch Faktoren wie Antwortlatenz, Sprachausgabequalität und die Synchronisation zwischen visueller Darstellung und Audiowiedergabe beeinflusst. Die Optimierung dieser Aspekte erforderte iteratives Anpassen und Testen über mehrere Entwicklungszyklen hinweg.

4 Produktbeschreibung

Pablo ist ein kompakter Desk-Companion-Bot, der zur Unterstützung von Nutzenden im Alltag entwickelt wurde. Das Produkt wurde als physisch präsentes Assistenzsystem konzipiert, das im direkten Umfeld der Nutzenden eingesetzt werden kann und eine niedrigschwellige Interaktion ermöglicht. Im Unterschied zu rein softwarebasierten Assistenzlösungen basiert Pablo auf dem Ansatz eines physischen Systems, das sprachliche, visuelle und funktionale Elemente in einem Produkt vereint. Ziel des entwickelten Systems ist es, spontane Aufgaben, Fragen und Erinnerungen entgegenzunehmen und in einer möglichst natürlichen Form zu verarbeiten.

Die Interaktion mit dem System erfolgt primär sprachbasiert. Dadurch kann die Nutzung ohne klassische Eingabemedien wie Tastatur, Maus oder Smartphone erfolgen. Ergänzt wird die sprachliche Interaktion durch visuelle Rückmeldungen über ein Display. Durch die Kombination aus physischer Präsenz, visueller Darstellung und sprachlicher Kommunikation wurde ein System entwickelt, das nicht ausschließlich funktionale Assistenzaufgaben erfüllt, sondern zugleich als Companion im persönlichen Umfeld wahrgenommen werden kann.

Ein wesentliches Merkmal von Pablo ist der modulare und personalisierbare Systemaufbau. Die entwickelte Architektur erlaubt es, einzelne Funktionen bedarfsgerecht zu ergänzen. Auf diese Weise kann das Produkt flexibel an unterschiedliche Anforderungen und Nutzungsszenarien angepasst werden. Die Personalisierbarkeit beschränkt sich damit nicht nur auf den Funktionsumfang, sondern betrifft grundsätzlich auch die Ausrichtung des Systems auf verschiedene Anwendungskontexte. Dadurch ist Pablo nicht ausschließlich für ein einzelnes Einsatzszenario ausgelegt, sondern kann perspektivisch für unterschiedliche Zielgruppen eingesetzt werden, beispielsweise als organisatorische Alltagshilfe, als Lernunterstützung oder als individuell konfigurierbarer Companion im privaten Umfeld. Diese Offenheit und Erweiterbarkeit stellt ein zentrales Unterscheidungsmerkmal gegenüber geschlossenen Sprachassistenzsystemen dar, deren Funktionsumfang und Erscheinungsbild in der Regel nur eingeschränkt angepasst werden können.

4.1 Kernfunktionen

Die Kernfunktionen von Pablo orientieren sich an typischen Anforderungen des Alltags und bilden die Grundlage für einen funktionalen Prototyp.

Die Aktivierung des Systems erfolgt wakeword-basiert. Dadurch verbleibt Pablo im Bereitschaftszustand, bis eine definierte sprachliche Aktivierung erkannt wird. Dieses Interaktionsprinzip wurde gewählt, um die Bedienung möglichst natürlich zu gestalten und gleichzeitig eine gezielte Nutzung des Systems zu ermöglichen. Nach Erkennung des Wakewords richtet Pablo seinen Bildschirm automatisiert in die Richtung aus, aus der die Stimme detektiert wurde. Nach der Aktivierung erfolgt die weitere Interaktion sprachlich, sodass Nutzende Anfragen und Befehle direkt formulieren können.

Eine zentrale Funktion ist die Beantwortung allgemeiner Fragen mithilfe eines Large Language Models. Dadurch ist Pablo in der Lage, auf sprachlich formulierte Anfragen flexibel und kontextbezogen zu reagieren. Darüber hinaus verfügt Pablo über konkrete Funktionsaufrufe für

Anwendungen. Dazu zählen das Erstellen und Verwalten von Notizen, Erstellen von Kalendereinträgen, das Setzen von Timern und aktuelle Wetterabfragen. Diese Funktionen sind so gewählt, dass sie einen unmittelbaren praktischen Nutzen im täglichen Gebrauch bieten und gleichzeitig die modulare Erweiterbarkeit des Systems demonstrieren.

Insgesamt bilden diese Kernfunktionen die technische und konzeptionelle Basis des Produkts und schaffen eine Grundlage für weitere Entwicklung.

4.2 Softwarearchitektur

Die Softwarearchitektur von Pablo basiert auf einer modularen, ereignisgesteuerten Python-Struktur mit Trennung zwischen Ein- und Ausgabe, Dialogverarbeitung sowie Hardwaresteuerung. Beim Start werden zunächst die Systemkonfiguration, die verfügbaren Plugin-Module und die visuelle Prozessdarstellung initialisiert. Anschließend wird der Wakeword- und Richtungsdetektor konfiguriert, der als Trigger-Komponente für die Interaktionsschleife fungiert. Die aus der Wakeword-Analyse abgeleitete Schallrichtung wird unmittelbar auf die Servosteuerung übertragen, sodass sich die Anzeigeeinheit in Richtung der sprechenden Person ausrichtet.

Das System arbeitet mit einer Hauptschleife mit vier logisch getrennten Verarbeitungsphasen: (1) akustische Aktivierung, Richtungsschätzung der Stimme und Anpassung der Servoposition, (2) Spracherkennung mit Speech-to-Text (STT), (3) inhaltliche Verarbeitung über ein Large Language Model (LLM) mit optionaler Modulausführung und (4) Sprachausgabe mit Text-to-Speech (TTS). Die einzelnen Phasen sind funktional entkoppelt und über definierte Funktionsaufrufe miteinander verbunden, wodurch die Austauschbarkeit einzelner Komponenten (z. B. STT- oder TTS-Backend) gewährleistet wird.

Ein zentrales Architekturelement ist die Plugin-Schnittstelle. Über den Loader werden externe Funktionsmodule dynamisch eingebunden und dem LLM-Kontext bereitgestellt. Durch den bereitgestellten Kontext kann das System zwischen generischer Antwortgenerierung und Modulausführung unterscheiden. Dies ermöglicht eine Erweiterung des Basissystem, ohne dass der Kernprozess verändert werden muss.

Parallelität wird gezielt für zeitkritische Teilaufgaben genutzt. Die Servosteuerung erfolgt asynchron in einem separaten Thread, sodass die Audioverarbeitung nicht blockiert wird. Zusätzlich wird die Zustandsumschaltung der Visualisierung während der Sprachausgabe zeitversetzt und threadbasiert umgesetzt, um eine konsistente Synchronisation zwischen auditivem und visuellem Feedback zu erreichen. Fehlerfälle werden über Ausnahmebehandlung auf Schleifenebene abgefangen.

4.2.1 (1) Aktivierung des Systems

4.2.2 (2) Spracherkennung (STT)

4.2.3 (3) LLM-Verarbeitung

4.2.4 (4) Sprachausgabe (TTS)

4.3 Hardwareaufbau

Die Auswahl der Hardwarekomponenten erfolgte unter Abwägung von Kosten, Kompaktheit, Integrationsfähigkeit und Community-Unterstützung. Als zentrale Recheneinheit wurde ein Raspberry Pi gewählt, der aufgrund seiner kompakten Bauform, breiten Softwarekompatibilität und umfangreichen Community-Dokumentation eine geeignete Plattform für ein Embedded-Assistenzsystem darstellt. Für die Audioerfassung wurde der ReSpeaker 2-Mics Pi HAT eingesetzt, der neben der Sprachaufnahme auch eine richtungsbasierte Schallquellenortung ermöglicht. Die visuelle Ausgabe erfolgt über ein Waveshare 4-Zoll-IPS-Display, das direkt am Raspberry Pi betrieben wird. Als Audioausgabe dient ein 5 W / 8 Ω-Lautsprecher, der über einen separaten Audioverstärker angesteuert wird. Die Stromversorgung des Verstärkers erfolgt über einen Arduino Uno, der zugleich als dedizierte Steuereinheit für die Energieversorgung der Audiokomponente fungiert. Für die mechanische Ausrichtung des Kopfteils, das aus Display und Gehäuseoberteil besteht, wird ein Servomotor eingesetzt, der über einen Arduino Nano angesteuert und seriell vom Raspberry Pi kontrolliert wird. Durch diese Aufteilung der Steuerungsaufgaben auf mehrere Mikrocontroller konnte die Echtzeitfähigkeit der mechanischen und audiovisuellen Komponenten sichergestellt werden, ohne die Hauptrecheneinheit zusätzlich zu belasten.

Das Gehäuse wurde als 3D-gedruckter Baumstamm gestaltet und nimmt sämtliche Hardwarekomponenten auf. Diese Formgebung unterstreicht den Companion-Charakter von Pablo und schafft eine visuelle Abgrenzung gegenüber rein technischen Geräten.

5 Begründung zentraler Entwicklungsentscheidungen

Diesen Abschnitt würde ich ausdrücklich separat machen, weil die Aufgabenstellung genau das fordert. Hier beschreibt ihr nicht nur Entscheidungen, sondern immer auch:Alternative – Auswahl – Begründung

5.1 Entscheidung für einen physischen Companion statt reinem Softwareassistenten

höhere Präsenz stärkere emotionale Bindung direkterer Produktcharakter

5.2 Entscheidung für modular erweiterbare API-Struktur

neue Funktionen können nachträglich ergänzt werden hohe Anpassbarkeit an verschiedene Zielgruppen Open-Source-Potenzial

5.3 Entscheidung für Raspberry-Pi-basierte Hardware

gute Verfügbarkeit geeignet für Prototyping Schnittstellenvielfalt ausreichend für lokale Koordination der Komponenten

5.4 Entscheidung für Wakeword- und Sprachsteuerung

niedrigschwellige Interaktion natürlicher Zugriff hands-free Nutzung

5.5 Entscheidung für eine visuelle Persona

Wiedererkennungswert freundlichere Interaktion Abgrenzung von rein technischen Assistentensystemen

5.6 Entscheidung für ein kompaktes Display

lokales Feedback Statusanzeige Charakterdarstellung von Pablo

6 Ergebnis und kritische Einordnung

Inhalt Was wurde erreicht? Inwiefern erfüllt der Prototyp die Anforderungen? Wo liegen aktuelle Grenzen?

Mögliche Punkte funktionsfähiger Prototyp eines modularen Desk-Companion-Bots erfolgreiche Kombination von Hardware, Sprachinteraktion und API-Modulen gute Grundlage für Erweiterungen Grenzen z. B.: Robustheit im Alltagsbetrieb Abhängigkeit von externen Diensten Systemstart und Hardwarestabilität begrenzte Rechenressourcen

Dazu passend Ein kurzer Ausblick ist hier sinnvoll: weitere Module verbesserte Gehäuseintegration robustere lokale Ausführung stärkere Personalisierung

Literatur

[1] M. B. Hoy, „Alexa, Siri, Cortana, and More: An Introduction to Voice Assistants," *Medical Reference Services Quarterly*, Jg. 37, Nr. 1, S. 81–88, 2018. DOI: 10.1080/02763869.2018.1404391

[2] V. Kepuska und G. Bohouta, „Next-Generation of Virtual Personal Assistants (Microsoft Cortana, Apple Siri, Amazon Alexa and Google Home)," *IEEE 8th Annual Computing and Communication Workshop and Conference (CCWC)*, S. 99–103, 2018. DOI: 10.1109/CCWC.2018.8301638

[3] W. X. Zhao u. a., „A Survey of Large Language Models," *arXiv preprint arXiv:2303.18223*, 2023.

[4] F. Bentley, C. Luvogt, M. Silverman, R. Wirasinghe, B. White und D. Lottridge, „Understanding the Long-Term Use of Smart Speaker Assistants," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, Jg. 2, Nr. 3, S. 1–24, 2018. DOI: 10.1145/3264901

[5] J. Li, „The Benefit of Being Physically Present: A Survey of Experimental Works Comparing Copresent Robots, Telepresent Robots and Virtual Agents,“ *International Journal of Human-Computer Studies*, Jg. 77, S. 23–37, 2015. DOI: 10.1016/j.ijhcs.2015.01.001

[6] T. Belpaeme, J. Kennedy, A. Ramachandran, B. Scassellati und F. Tanaka, „Social Robots for Education: A Review,“ *Science Robotics*, Jg. 3, Nr. 21, eaat5954, 2018. DOI: 10.1126/scirobotics.aat5954